MIT Virtual Source GaN HEMT (MVSG_CMC) Model v4.0.0 official release updates

18 March 2024

Lan Wei and Ujwal Radhakrishna

University of Waterloo

MVSG_CMC Model Update Brief Summary

- **Added**
  1. p-GaN Module functionality: high injection, reverse recombination voltage saturation, secondary reverse recombination, ohmic junction, regrouping of parameters, allowed ranges of parameters changed
  2. Gummel symmetry changes: new parameter *flaggum* uses new $xtanh(x)$ smoothing, new parameter *mmaxs* controls level of smoothing, default β=2, SAR & DAR source voltage uses smoothing function when *flaggum*=1
  3. QA tests: additional p-GaN elements (DC), gummel symmetry (DC)
  4. New parameter macros: MPRcc, MPIcc; many already existing parameters swapped over to new macros
- **Changed**
  1. *minr* allowed to be zero
  2. New p-GaN capacitance model: conserves charge, smoothing used when bias approaches built in potential, apply $csh0\ (new) = \frac{csh0\ (old)}{\sqrt{v_{csh0}}}$ to maintain same capacitance values in region of interest
  3. High injection formulation in Schottky diode forward mode calculation: new formulation ensures current is monotonic and behaves physically, additionally easier to extract parameters, parameters have same meaning however previous fits will need to be refit
  4. Parameters: reorganization of gate parameters representing functional grouping, rcapture & remission descriptions
- **Fixed**
  1. Charge not being conserved in p-GaN capacitance model
  2. Internal nodes tied off properly when *trapselect* = 1 or 2
  3. calc_capt function now returns output value once
  4. explim function extra nesting removed (does not affect model functionality)

MVSG_CMC Model Update Summary beta0_1 Details

- **V4.0.0beta0_1 – 11 May 2023 (Primary contributors: Jessica Chong)**

1. Updated smoothing function implementation with toggle
2. Default beta = 2
3. Updated to allow a minimum resistance of 0
4. Noise calculations for gate diodes updated to include gate current debiasing
5. Gate leakage parameters reorganized for better functional grouping
6. Updated rcapture and remission parameter descriptions for clarity
7. (see beta release document for more details in slides)

MVSG_CMC Model Update Summary beta0_2 Details

- **V4.0.0beta0_2 – 17 July 2023 (Primary contributors: Ryan Fang)**

1. Additional parameters for p-GaN module diode: high injection, reverse recombination voltage saturation, secondary reverse recombination, ohmic junction
2. Order and grouping of p-GaN parameters
3. csh0 parameter changed from “oz” to “cz”
4. Additional QA test
5. (see beta release document for more details in slides)

MVSG_CMC Model Update Summary beta0_3 Details

- **V4.0.0beta0_3 – 6 September 2023 (Primary contributors: Ryan Fang)**

1. Removed old p-GaN capacitance “csh” calculation and replaced with a charge conserving one
2. Domain of charge calculation extended with Taylor series – option to choose order between 0 and 5
3. “fc” parameter added to determine when transition to Taylor series begins
4. Built in potential of p-GaN Schottky junction parameter “vcsh0” changed from nb to oz
5. p-GaN zero bias junction capacitance parameter “csh0” units updated
6. Branch directions in p-GaN module all aligned to be the same
7. Fixed some local variable initialization to zero, renamed some local variables
8. vsch = V(gi2p,gi2) added due to many calls to it
9. Converted some integers constants in p-GaN module into floats
10. Changed trapselect to MPIcc
11. (see beta release document for more details in slides)

MVSG_CMC Model Update Summary beta0_4 Details

- **V4.0.0beta0_4 – 20 November 2023 (Primary contributors: Ryan Fang, Johan Alant)**

1. SAR and DAR source voltage computed with mmax if flaggum=1 to remove discontinuities in currents
2. New parameter *mmaxs* to allow user controllable smoothing for absfunc and mmax calls
3. MPRcc macro added – used by *fracs, fracd, frac_pgan, ohmicratio*
4. Reworked high injection formulation to allow easier modeling and more accurate physical representation
   a. Additional current calculated to represent high injection current beyond *vgsat{s/d/_pgan}*
   b. Original fermi function used to smooth ideality factor now used to smooth between normal and above current
   c. Default smoothing parameter *alphag{s/d/_pgan}* changed to 1.0 (from 10.0)
5. Trapping internal nodes tied off when trapselect = 1 or 2
6. explim function extra nested if statement removed
7. (see beta release document for more details in slides)

MVSG_CMC Model Update Summary beta1 Details

- **V4.0.0beta1 – 15 January 2024 (Primary contributors: Ryan Fang)**

1. Fixed calc_capt function returning output value twice

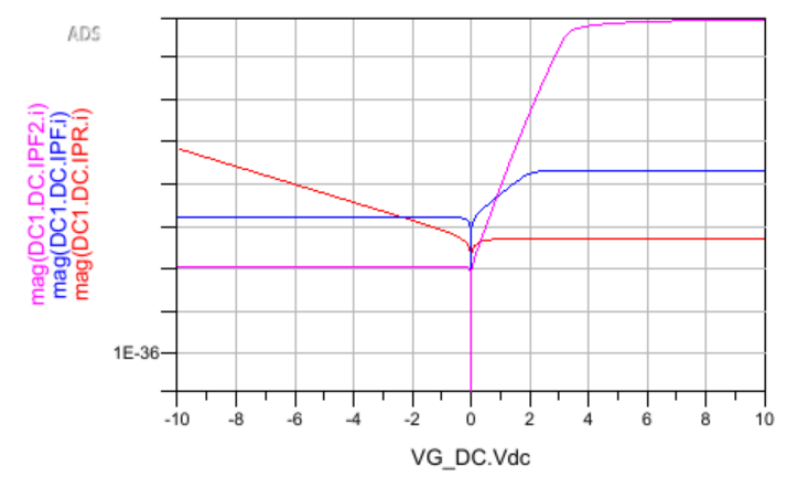
MVSG_CMC Model Update Summary beta2 Details

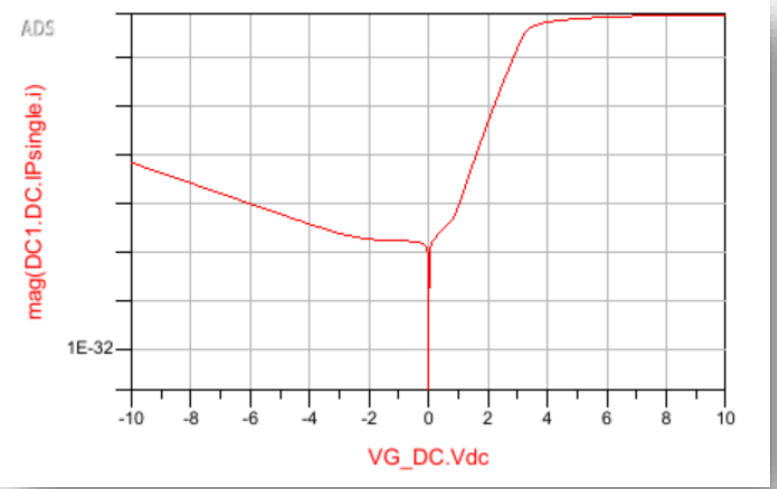
- **V4.0.0beta2 – 15 February 2024 (Primary contributors: Ryan Fang)**

1. Update comments prepared for official release
2. New & Revised QA tests to include testing of several default zero parameters

p-GaN Diode Additional Functionality

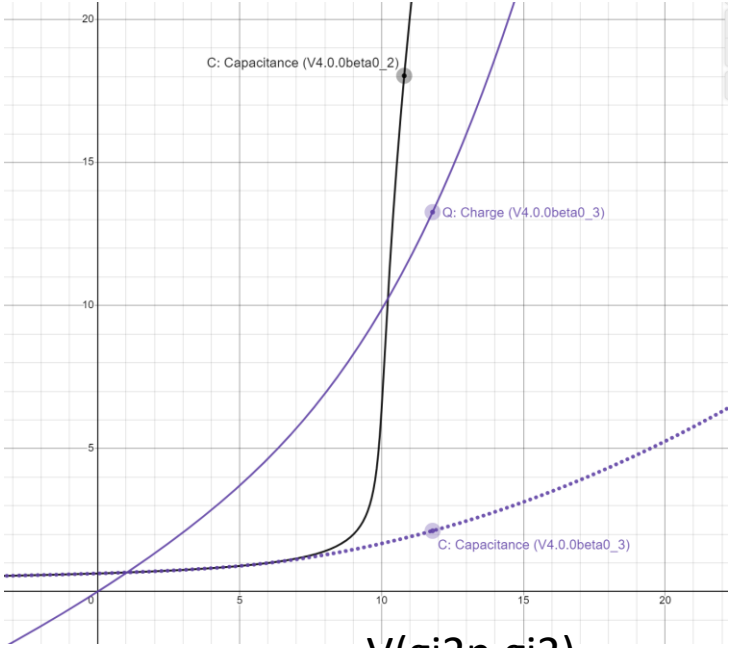
- Left shows IV of p-GaN diode forward component (red, high injection not shown but may now be enabled), reverse recombination component with voltage saturation (blue), and secondary reverse recombination component (magenta, voltage saturation not enabled, but mathematical clamping triggered to prevent extremely large values)
- Right shows total IV of p-GaN diode
- (Note: diode is connected backwards)





p-GaN New Capacitance Model

- In V3.2.0, during transient simulation, since each time step uses a fixed value for the bias dependent capacitance, despite bias being a continuously changing function
  - Results in charge not being conserved
- In V4.0.0beta1, state function “qcsh” is used to compute charge directly, resulting in no dependence on time step size, instead only on initial and final bias.
  - Derived capacitance value is identical as before, with exception of mathematical domain extending region (not part of region of interest)



V(gi2p,gi2)

p-GaN New Capacitance Model Equations

- In V3.2.0, computation of capacitance, and then current
  - $c_{sh} = \frac{c_{sh0} \cdot w \cdot n_{gf}}{\sqrt{v_{csh0} + V(gi2,gi2p)}}$
  - $I(gi2, gi2p) < + c_{sh} * ddt(V(gi2, gi2p))$
  - mmax function used to ensure c_{sh} calculation does not blow up
- In V4.0.0beta1, computation of charge, and then current

1. $q_{csh} = 2 \cdot c_{sh0} \cdot w \cdot n_{gf} \cdot v_{csh0} \left(1 - \sqrt{1 - \frac{V(gi2p,gi2)}{v_{csh0}}}\right)$

  - $I(gi2p, gi2) < + ddt(q_{csh})$
  - Remark: q_{csh0} calculation is off by a factor of $\sqrt{v_{csh0}}$ compared to V4.0.0beta0_2. The new formulation is backed in literature and results in c_{sh0} to have more ordinary units of Farads per meter. All users will need to apply $csh0\ (new) = \frac{csh0\ (old)}{\sqrt{v_{csh0}}}$ to maintain same capacitance values in region of interest
- In V4.0.0beta1, function domain boundary (i.e. capacitance blow up) is handled by Taylor series extension
  - fc*vcsh0 determines voltage at which this occurs
  - pgancshorder parameter determines order of precomputed Taylor series (integer values between 0 – 5)
  - Formulation:
  - $q_{csh} = 2 \cdot c_{sh0} \cdot w \cdot n_{gf} \cdot v_{csh0} \cdot \sum_{n=0}^{\text{pgancshorder}} \left[\frac{\left(1 - \sqrt{1 - \frac{V(gi2p,gi2)}{v_{csh0}}}\right)^{(n)} \Big|_{V(gi2p,gi2) = f_c * v_{csh0}}}{n!} (V(gi2p, gi2) - f_c \cdot v_{csh0})^n \right]$

Gummel Symmetry Fix

The discontinuity was traced back to the two following implementations

- Issue 1: Smoothing function (mainly used for transitioning between triode and saturation)

$$\frac{1}{(1+x^{\beta})^{1/\beta}}$$

- Issue 2: Abs/max function

$$absfunc = \sqrt{x^2 + 4.0 * 1e^{-5}}$$
$$mmax = 0.5(\mathrm{x} + \mathrm{y} + \sqrt{(x-y)^2 + 4.0 * 1e^{-5}})$$

Solution for Issue 1: Setting $\beta = 2$

Solution for Issue 2: New abs/max function

$$absfunc = x * \tanh(k_{max} * x)$$
$$mmax = 0.5(x + y + (x-y) * \tanh(k_{max} * (x-y)))$$

Gummel Symmetry Fix

```
analog function real absfunc;
    input x;
    real x;
    begin
        if (flaggum==0) begin
            absfunc=sqrt( x * x + 4.0 * 1e-5  );
        end else begin
            absfunc= x * tanh( x );
        end
    end
endfunction
```

Old

```
analog function real mmax;
    input x,y;
    real x,y;
    begin
        if (flaggum==0) begin
            mmax=0.5 * ( x + y + sqrt( ( x - y ) * ( x - y ) + 4.0 * 1e-5 ));
        end else begin
            mmax=0.5 * ( x + y + ( x - y ) * tanh(( x - y )));
        end
    end
endfunction
```

New

Gummel Symmetry Fix Usage

- Setting $\beta = 2$
  - Users can still choose a non-even value of beta for the flexibility of curve fitting, unless applications where continuity in the 3rd order derivative or higher is critical
  - Not backward compatible

- Flaggum toggle
  - Allows users to choose between the old and new smoothing functions
  - For applications where continuity in the 3rd order derivative or higher is critical, users should choose flaggum=1
  - For other applications, the original smoothing functions are recommended for computational speed
  - No impact on backward compatibility

Gummel Symmetry Fix Usage

- Users who would like differentiability of the current at VDS=0V should choose an even beta and use the new smoothing function
- The new smoothing function is found in absfunc and mmax and can be using flagggum=0 for default function and flaggum=1 for new function

```
`MPIsw(flaggum,  0,        "",        "Flag parameter for smoothing function in absfunc and mmax:
xtanhx smoothing function used if flaggum=1 or original implementation if flaggum=0")
```

User-case Recommendations

- **RF amplifier** – higher order harmonics around VDS=0 matters, transitioning between triode/saturation less concerned
  - Set beta = 2
  - Use new max/abs function

- **Power converter** – transitioning between triode/saturation matter, higher order harmonic less concerned
  - Adjust beta to get the best fit between triode/saturation
  - Use old max/abs function

SAR & DAR Gummel Symmetry Discontinuity Fix

- Smoothed transitions of SAR and DAR implicit-gate-to-source voltages so that the regions pass Gummel Symmetry Tests.

The if-statements cause discontinuities in derivatives of vsars and vdars at the point where they switch, which introduce discontinuities in derivatives of the drain current. Using “mmax” smooths the transition.

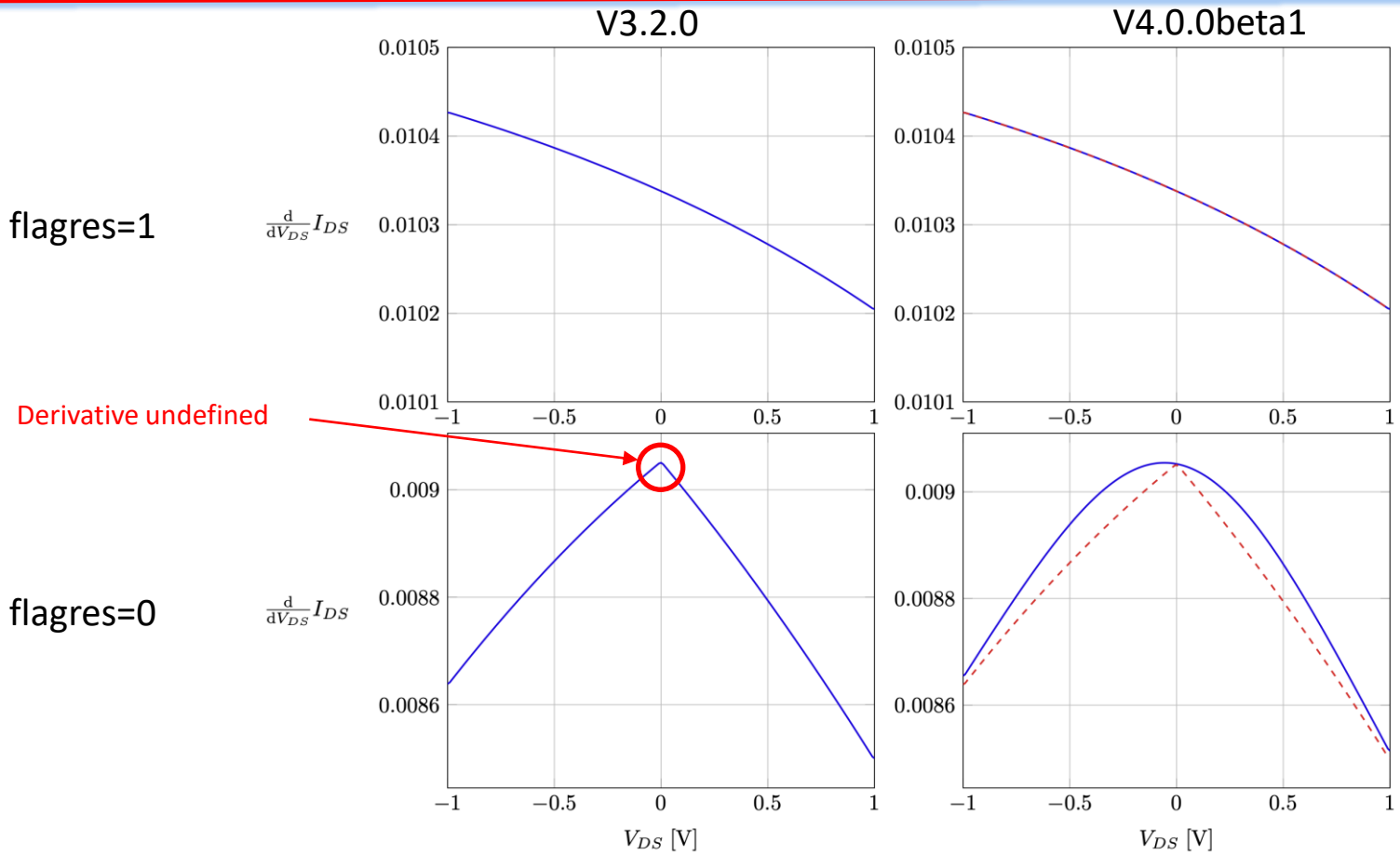
```
// Determine drain-source and gate-source voltage for SAR transistor
if (type * V(src, d) <= type * V(src, s)) begin
   vsars = type * V(src, s);
end else begin
   vsars = type * V(src, d);
end
vigs  = vtors + 1.0 / (rsh * cgrs * mu0);
vdsrs = type * V(fps4, src);
vgsrs = (vigs - vsars);
```

```
if (flaggum == 0) begin
   if (type * V(src, d) <= type * V(src, s)) begin
      vsars = type * V(src, s);
   end else begin
      vsars = type * V(src, d);
   end
end else begin
   vsars = mmax(type * V(src, d), type * V(src, s), mmaxs);
end
```

```
// Determine drain-source and gate-source voltage for DAR transistor
if (type * V(fp4, d) <= type * V(fp4, s)) begin
   vdars = type * V(fp4, s);
end else begin
   vdars = type * V(fp4, d);
end
vigd  = vtord + 1.0 / (drsht * rsh * cgrd * mu0);
vdsrd = type * V(drc, fp4);
vgsrd = (vigd - vdars);
```

```
if (flaggum == 0) begin
   if (type * V(fp4, d) <= type * V(fp4, s)) begin
      vdars = type * V(fp4, s);
   end else begin
      vdars = type * V(fp4, d);
   end
end else begin
   vdars = mmax(type * V(fp4, d), type * V(fp4, s), mmaxs);
end
```

SAR & DAR Gummel Symmetry Discontinuity Fix Comparison



The amount by which the new implementation deviates from the case where flaggum==0 is determined by “mmax” and “absfunc” and can be reduced by utilizing the new “mmaxs” smoothing parameter.

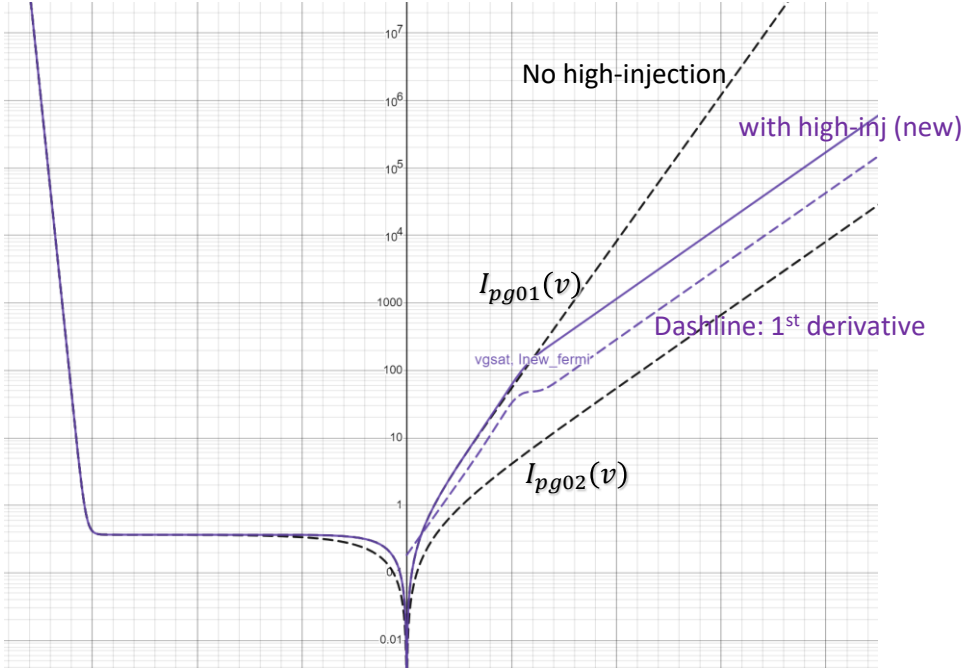
New Schottky Diode High Injection Formulation

- New implementation: smoothing between I_{pg01} (V_g<Vgsat)

  and $\frac{I_{pg01}(v_{gsat})}{I_{pg02}(v_{gsat})} I_{pg02}(v_{gin})$ (V_g>Vgsat)

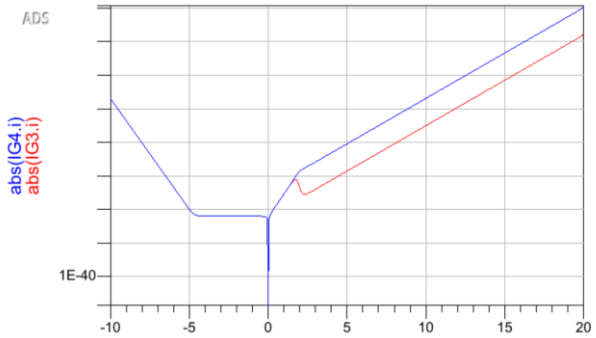
$$I_g(v_{gin}) = F_{f,vg} I_{pg01}(v_{gin}) + (1 - F_{f,vg}) I_{hinj}(v_{gin})$$

$F_{f,vg}$ is a fermi smoothing function around Vgsat



New Schottky Diode High Injection Simulation

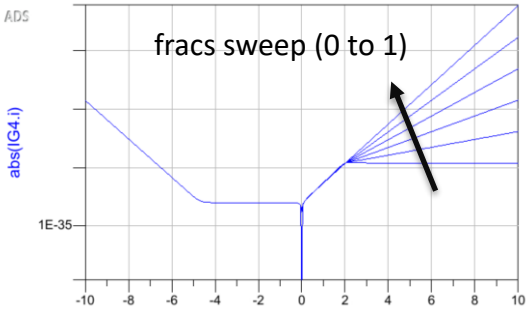
- beta0_3 vs beta0_4

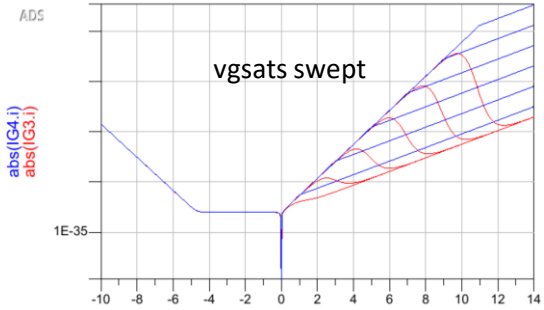


Beta0_3 (Red) shows non-monotonic behavior at the on-set of high-injection effect

Beta0_4 (blue) I_g –V_g now monotonic (physical)

High-injection behaviors can be flexibly adjusted





New QA Tests

- Schottky p-GaN DC test with all additional gate current sections enabled
- Gummel symmetry test

```
304  //  VI.b Nominal IdVg sweep with Schottky pGaN module actived and e-mode power device parameter set and all sections of gate current
305  test                        Test23_dcSweep_idvg_pGaN_otherSections
306  float                       dt
307  temperature                 -55 27 150
308  biases                      V(s)=0 V(b)=0
309  biasList                    V(d)=10,20,100
310  biasSweep                   V(g)=0,5,0.1
311  outputs                     I(d) I(s) I(g) I(b)
312  instanceParameters          w=1.0e-7 l=1.0e-6
313  modelParameters             parameters/poweremodeParameters
314  modelParameters             flagpgan=1 frac_pgan=0.4 vgsatq_pgan=5 pganrecmod=1
315
316  // VII. DC Gummel Symmetry test
317
318  //  VI.a Nominal IdVd sweep with Gummel Symmetry module actived
319  test                        Test24_dcSweep_idvd_gummel
320  float                       dt
321  temperature                 -55 27 150
322  biases                      V(s)=0 V(b)=0
323  biasList                    V(g)=-1,0,1
324  biasSweep                   V(d)=-1,1,0.5
325  outputs                     I(d) I(s) I(g) I(b)
326  instanceParameters          w=30e-6 l=250.0e-9
327  modelParameters             parameters/dmodeParameters
328  modelParameters             flaggum=1 igmod=0 beta=2
329
```

New & Revised QA Tests for default zero parameters

- Parameters include tempcos, transport, access region, FP, minimum capacitance

```
//  I.f Non-zero secondary parameter values
test                    Test6_dcSweep_idvd_secondaryParameter
float                   dt
temperature             27 150
biases                  V(s)=0 V(b)=0
biasList                V(g)=-5,-4,-3,-2,-1,0,1,2,3
biasSweep               V(d)=1e-02,20.01,0.5
outputs                 I(d) I(s) I(g) I(b)
instanceParameters      w=1.0e-3 l=250e-9
modelParameters         parameters/dmodeParameters
modelParameters         rctl=1e-2 rct2=1e-4 delta2=1e-3 nd=0.01 lambda=2e-3 vtheta=0.01 mtheta=0.01
modelParameters         ndrs=0.01 vthetars=0.01 mthetars=0.01 vthetard=0.01 mthetard=0.01
modelParameters         fracig=1 kbdgates=1 kbdgated=1 igrecmod=1 lovg=10e-9
```

```
//  I.e Non-zero secondary parameters values
test                    Test31_acSweep_vd_fringingCaps
float                   dt
temperature             -55 27 150
biases                  V(s)=0 V(b)=0
biasList                V(g)=-5,-2,0
biasSweep               V(d)=1e-2,20.01,0.5
outputs                 G(d,g) G(d,d) C(g,s) C(g,d) C(g,g)
instanceParameters      w=1.0e-3 l=250e-9
modelParameters         parameters/dmodeParameters
modelParameters         tcg=0.001 cofsm=1e-15 cofdm=1e-15 cofdsm=1e-15 cofdsubm=1e-15 cofssubm=1e-15 cofgsubm=1e-15
```

```
//  II.h Four source-side FPs turned on
test                    Test14_dcSweep_SS_4fps
float                   dt
temperature             -55 27 150
biases                  V(s)=0 V(b)=0 V(g)=-3
//biasList                V(g)=-3,-2,-1,0,1,2,3
biasSweep               V(d)=1e-2,20.01,0.5
outputs                 I(d) I(s) I(g) I(b)
instanceParameters      w=1.0e-3 l=250e-9
modelParameters         parameters/dmodeParameters
modelParameters         lgfps1=1e-6 lgfps2=2e-6 flagfps1=1 flagfps2=1 lgfps3=3e-6 lgfps4=2e-6 flagfps3=1 flagfps4=1 flagres=1
modelParameters         cfps1s=1e-15 cbfps1=1e-15 tcbfps1=0.1 deltalfps1=0.1 ndfps1=0.01 vthetafps1=0.1 mthetafps1=0.1
modelParameters         cbfps2=1e-15 tcbfps2=0.01 deltalfps2=0.1 ndfps2=0.01 vthetafps2=0.01 mthetafps2=0.01
modelParameters         tccfps3=0.1 cbfps3=1e-15 tcbfps3=0.1 deltalfps3=0.1 ndfps3=0.01 vthetafps3=0.1 mthetafps3=0.1
modelParameters         tcgfps4=0.1 tccfps4=0.1  cbfps4=1e-15 tcbfps4=0.1 deltalfps4=0.1 ndfps4=0.01 vthetafps4=0.01 mthetafps4=0.01

//  II.i Four drain-side FPs turned on
test                    Test15_dcSweep_DS_4fp
float                   dt
temperature             -55 27 150
biases                  V(s)=0 V(b)=0 V(g)=-3
//biasList                V(g)=-3,-2,-1,0,1,2,3
biasSweep               V(d)=1e-2,20.01,0.5
outputs                 I(d) I(s) I(g) I(b)
instanceParameters      w=1.0e-3 l=250e-9
modelParameters         parameters/dmodeParameters
modelParameters         lgfp1=1e-6 lgfp2=2e-6 flagfp1=1 flagfp2=1 lgfp3=3e-6 lgfp4=2e-6 flagfp3=1 flagfp4=1 flagres=1
modelParameters         cbfp1=1e-15 tcbfp1=0.1 deltalfp1=0.1 ndfp1=0.01 vthetafp1=0.1 mthetafp1=0.1
modelParameters         cbfp2=1e-15 tcbfp2=0.1 deltalfp2=0.1 ndfp2=0.01 vthetafp2=0.01 mthetafp2=0.01
modelParameters         tccfp3=0.1 cbfp3=1e-15 tcbfp3=0.1 deltalfp3=0.1 ndfp3=0.01 vthetafp3=0.1 mthetafp3=0.1
modelParameters         tcgfp4=0.01 tccfp4=0.1 cbfp4=1e-15 tcbfp4=0.1 deltalfp4=0.1 ndfp4=0.01 vthetafp4=0.01 mthetafp4=0.01
```

```
//  II.e AC- Vd sweep with 4 FPs turned on
test                    Test36_acSweep_vd_allfourFP_turnedon
float                   dt
temperature             27
biases                  V(s)=0 V(b)=0
biasList                V(g)=-5,-4
biasSweep               V(d)=1e-2,100.01,5
outputs                 C(g,s) C(g,d) C(g,g)
instanceParameters      w=1.0e-3 l=250e-9
modelParameters         parameters/dmodeParameters
modelParameters         lgfp1=1e-6 flagfp1=1 lgfp2=2e-6 flagfp2=1 lgfp3=1e-6 flagfp3=0 lgfp4=2e-6 flagfp4=0 igmod=0 flagres=1
modelParameters         minc=1e-18
```

New Parameter Macros

- MPRcc and MPIcc added according to VerilogA code standards

```
// Define macros for parameters
`define MPRnb(nam,def,uni,         des) (*units=uni,                    desc=des*) parameter real    nam=def;
`define MPRco(nam,def,uni,lwr,upr,des) (*units=uni,                    desc=des*) parameter real    nam=def from[lwr:upr);
`define MPRcc(nam,def,uni,lwr,upr,des) (*units=uni,                    desc=des*) parameter real    nam=def from[lwr:upr];
`define MPRcz(nam,def,uni,         des) (*units=uni,                    desc=des*) parameter real    nam=def from[  0:inf);
`define MPRoz(nam,def,uni,         des) (*units=uni,                    desc=des*) parameter real    nam=def from(  0:inf);
`define MPIsw(nam,def,uni,         des) (*units=uni,                    desc=des*) parameter integer nam=def from[  0:  1];
`define MPIty(nam,def,uni,         des) (*units=uni,                    desc=des*) parameter integer nam=def from[ -1:  1] exclude 0;
`define MPIco(nam,def,uni,lwr,upr,des) (*units=uni,                    desc=des*) parameter integer nam=def from[lwr:upr);
`define MPIcc(nam,def,uni,lwr,upr,des) (*units=uni,                    desc=des*) parameter integer nam=def from[lwr:upr];
`define IPRnb(nam,def,uni,         des) (*units=uni, type="instance",  desc=des*) parameter real    nam=def;
`define IPRoz(nam,def,uni,         des) (*units=uni, type="instance",  desc=des*) parameter real    nam=def from(  0:inf);
`define IPIco(nam,def,uni,lwr,upr,des) (*units=uni, type="instance",  desc=des*) parameter integer nam=def from[lwr:upr);
```

New p-GaN Parameters

- Additional parameters for gate current sections, controllability of capacitance during blow up when reaching built in potential, ohmic junction

```
//p-Gan junction parameters
`MPIsw(flagpgan,      0,       "",       "Flag parameter for pGan calculations 0=off, 1=on")
`MPRcz(pg_param_pgan,0.05,     "1/V",    "pGan forward diode inverse of ideality factor")
`MPRcz(ij_pgan,       9.0e-10,"A/m",    "pGan forward diode leakage current normalized to width")
`MPRcz(pgsrec_pgan,   0.013,  "",       "pGan diode inverse of ideality factor reverse recombination")
`MPRcz(irec_pgan,     4.5e-9, "A/m",    "pGan reverse leakage current normalized to width")
`MPRnb(vcsh0,         2.0,    "V",      "Built in potential of pGan Schottky junction")
`MPRoz(csh0,          6e-8,             "F*V^(1/2)/m","Fitting parameter for pGan Schottky junction capacit
```

```
//p-Gan junction parameters
`MPIsw(flagpgan,      0,       "",       "Flag parameter for pGan calculations 0=off, 1=on")

`MPRcz(pg_param_pgan,0.05,     "1/V",    "pGan forward diode inverse of ideality factor")
`MPRcz(ij_pgan,       2e-5,    "A/m",    "pGan forward diode leakage current normalized to width")
`MPRcz(vgsat_pgan,    3,       "V",      "pGan high injection effect")
`MPRcc(frac_pgan,     0.4,     "", 0, 1, "pGan fractional change in ideality factor due to high injection")
`MPRoz(alphag_pgan,   1.0,     "",       "pGan high injection smoothing parameter")

`MPRcz(pgsrec_pgan,   0.5,     "",       "pGan diode inverse of ideality factor reverse recombination")
`MPRcz(irec_pgan,     1e-21,"A/m",      "pGan reverse leakage current normalized to width")
`MPRoz(vgsatq_pgan,   2e4,     "V",      "pGan mimics depletion saturation")
`MPRoz(betarec_pgan, 1,        "",       "pGan linear to saturation parameter")

`MPIsw(pganrecmod,    0,       "",       "Flag parameter to turn on secondary gate-recombination current for p-Gan 0=off,
`MPRcz(pgsrec_pgan2,  0.5,     "",       "Secondary pGan diode inverse of ideality factor reverse recombination")
`MPRcz(irec_pgan2,    1e-21,"A/m",      "Secondary pGan reverse leakage current normalized to width")
`MPRoz(vgsatq_pgan2,  2e4,     "V",      "Secondary pGan mimics depletion saturation")
`MPRoz(betarec_pgan2,1,        "",       "Secondary pGan linear to saturation parameter")

`MPRoz(vcsh0,         2.0,     "V",      "Built in potential of pGan Schottky junction")
`MPRcz(csh0,          6e-8,    "F/m",    "pGan zero bias Schottky junction capacitance")
`MPRco(fc,            0.5,     "",  0, 1, "Fractional change in pGan charge between 0 and root domain boundary before Tayl
`MPIcc(pgancshorder,  2,       "",  0, 5, "Order of Taylor series to compute pGan charge beyond fc*vcsh0")
`MPRcz(rsch0,         0,       "Ohm*m",  "Ohmic contact for the Schottky junction normalized to width")
`MPRcc(ohmicratio,    0,       "m", 0, 1, "Fraction of device width that is ohmic, wsch = w * ohmicratio")
```

Smoothing Functions

- New formulation selectable by setting *flaggum*=1
- *mmaxs* parameter controls level of smoothing

```
analog function real absfunc;
    input x;
    real x;
    begin
        absfunc=sqrt( x * x + 4.0 * 1e-5  );
    end
endfunction

analog function real mmax;
    input x,y;
    real x,y;
    begin
        mmax=0.5*( x + y + sqrt( ( x - y ) * ( x - y ) + 4.0 * 1e-5 ));
    end
endfunction
```

```
analog function real absfunc;
    input x, s;
    real x, s;
    begin
        if (flaggum==0) begin
            absfunc=sqrt( x * x + s );
        end else begin
            absfunc= x * tanh( ( 1.0e-3 / s ) * x );
        end
    end
endfunction

analog function real mmax;
    input x,y,s;
    real x,y,s;
    begin
        if (flaggum==0) begin
            mmax=0.5 * ( x + y + sqrt( ( x - y ) * ( x - y ) + s ) );
        end else begin
            mmax=0.5 * ( x + y + ( x - y ) * tanh( ( 1.0e-3 / s ) * ( x - y ) ) );
        end
    end
endfunction
```

Update *calc_ig* Function

- To enable new high injection formulation

```
alpha_phit   = alphagin * phitin;
expphib      = pg_param1 / phitin * (- vjg);
t0           = explim( expphib );
expffvarg    = ( vgin - ( vgsatin - alphagin * alpha_phit / 2.0))/( alphagin * alpha_phit );
if (expffvarg > `M_MAXEXP) begin
    ffvgin   = 0.0;
end else if (expffvarg < -`M_MAXEXP) begin
    ffvgin   = 1.0;
end else begin
    ffvgin   = 1.0 / ( 1.0 + exp( expffvarg ));
end
pgin         = ( fracin * pg_paramin + ( 1.0 - fracin) * pg_paramin * ffvgin );

expbdarg1    = pbdgin * ( -vgin  - vbdgin ) + expphib;
expbdarg2    = -pbdgin * vbdgin + expphib;
expbd1       = explim( expbdarg1 );
expbd2       = explim( expbdarg2 );
iginbd       = ( expbd1 - expbd2 );

isdiodeout   = type * w * ngf * ijin * tfacdiodein;
expiforarg   = pgin / phitin * vgin + expphib;
expifor      = explim( expiforarg );
igindiode    = isdiodeout * ( expifor - ( kbdgatein * iginbd ) - t0 );

frecgin      = -vgin / pow(( 1.0 + pow( absfunc( vgin  / vgsatqin ), betarecin )), 1.0 / betarecin );
isrecout     = -type * w * ngf * irecin * tfacdiodein * 1.0;
expirevarg   = pgsrecin / phitin * frecgin;
expirev      = explim( expirevarg );
iginrec      = isrecout * ( expirev - 1.0 );
```

```
expphib      = pg_param1 / phitin * (- vjg);
t0           = explim( expphib );

expbdarg1    = pbdgin * ( -vgin  - vbdgin ) + expphib;
expbdarg2    = -pbdgin * vbdgin + expphib;
expbd1       = explim( expbdarg1 );
expbd2       = explim( expbdarg2 );
iginbd       = ( expbd1 - expbd2 );

isdiodeout   = type * w * ngf * ijin * tfacdiodein;
expiforarg   = pg_paramin / phitin * vgin + expphib;
expifor      = explim( expiforarg );
if (fracin == 1) begin
    igindiode    = isdiodeout * ( expifor - ( kbdgatein * iginbd ) - t0 );
end else begin
    expbdarg1_vgsat = pbdgin * ( -vgsatin  - vbdgin ) + expphib;
    expbd1_vgsat    = explim( expbdarg1_vgsat );
    iginbd_vgsat    = ( expbd1_vgsat - expbd2 );

    expiforarg_nohinj_vgsat = pg_paramin / phitin * vgsatin + expphib;
    expifor_nohinj_vgsat    = explim( expiforarg_nohinj_vgsat );
    igindiode_nohinj_vgsat  = ( expifor_nohinj_vgsat - ( kbdgatein * iginbd_vgsat ) - t0 );

    igindiode_nohinj   = isdiodeout * ( expifor - ( kbdgatein * iginbd ) - t0 );
    if (fracin > 0) begin
        pg_paramin_hinj = fracin * pg_paramin;

        expiforarg_hinj_vgsat   = pg_paramin_hinj / phitin * vgsatin + expphib;
        expifor_hinj_vgsat      = explim( expiforarg_hinj_vgsat );
        igindiode_hinj_vgsat    = ( expifor_hinj_vgsat - ( kbdgatein * iginbd_vgsat ) - t0 );

        expiforarg_hinj    = pg_paramin_hinj / phitin * vgin + expphib;
        expifor_hinj       = explim( expiforarg_hinj );
        igindiode_hinj_pre = isdiodeout * igindiode_nohinj_vgsat / igindiode_hinj_vgsat;
        igindiode_hinj     = igindiode_hinj_pre * ( expifor_hinj - ( kbdgatein * iginbd ) - t0 );
    end else begin
        igindiode_hinj = isdiodeout * igindiode_nohinj_vgsat;
    end
    alpha2_phit  = alphagin * alphagin * phitin;
    expffvarg    = ( vgin - ( vgsatin - alpha2_phit / 2.0))/ alpha2_phit;
    if (expffvarg > `M_MAXEXP) begin
        ffvgin   = 0.0;
    end else if (expffvarg < -`M_MAXEXP) begin
        ffvgin   = 1.0;
    end else begin
        ffvgin   = 1.0 / ( 1.0 + exp( expffvarg ));
    end
    igindiode = ffvgin * igindiode_nohinj + ( 1.0 - ffvgin ) * igindiode_hinj;
end

frecgin      = -vgin / pow(( 1.0 + pow( absfunc( vgin  / vgsatqin, mmaxs ), betarecin )), 1.0 / betarecin );
isrecout     = -type * w * ngf * irecin * tfacdiodein * 1.0;
expirevarg   = pgsrecin / phitin * frecgin;
expirev      = explim( expirevarg );
iginrec      = isrecout * ( expirev - 1.0 );
```

SAR & DAR Transistor Source Voltage Switch

- Smoothing the source voltage transition fixes Gummel symmetry issue with SAR and DAR

```
// Determine drain-source and gate-source voltage for SAR transistor
if (type * V(src,d)<= type * V(src,s)) begin
    vsars            = type * V(src,s);

end else begin
    vsars            = type * V(src,d);
end
vigs                 = vtors + 1.0 / (rsh * cgrs * mu0);
vdsrs                = type * V(fps4,src);
vgsrs                = (vigs - vsars);
```

```
// Determine drain-source and gate-source voltage for SAR transistor
if (flaggum == 0) begin
    if (type * V(src, d) <= type * V(src, s)) begin
        vsars = type * V(src, s);
    end else begin
        vsars = type * V(src, d);
    end
end else begin
    vsars            = mmax(type * V(src, d), type * V(src,s), mmaxs);
end
vigs                 = vtors + 1.0 / (rsh * cgrs * mu0);
vdsrs                = type * V(fps4,src);
vgsrs                = (vigs - vsars);
```

```
// Determine drain-source and gate-source voltage for DAR transistor
if (type * V(fp4,d)<= type * V(fp4,s)) begin
    vdars            = type * V(fp4,s);

end else begin
    vdars            = type * V(fp4,d);
end
vigd                 = vtord + 1.0 / (drsht * rsh * cgrd * mu0);
vdsrd                = type * V(drc,fp4);
vgsrd                = (vigd - vdars);
```

```
// Determine drain-source and gate-source voltage for DAR transistor
if (flaggum == 0) begin
    if (type * V(fp4, d) <= type * V(fp4, s)) begin
        vdars = type * V(fp4, s);
    end else begin
        vdars = type * V(fp4, d);
    end
end else begin
    vdars            = mmax(type * V(fp4, d), type * V(fp4, s), mmaxs);
end
vigd                 = vtord + 1.0 / (drsht * rsh * cgrd * mu0);
vdsrd                = type * V(drc,fp4);
vgsrd                = (vigd - vdars);
```

New p-GaN Module Code

- Contains new diode current sections, new charge model, and ohmic junction

```
if (flagpgan == 1) begin
    vsch          = type * V(gi2p,gi2);
    idsch         = calc_ig(idschsat,idschrec,vsch,phit,vgsat_pgan,alphag_pgan,frac_pgan,pg_param_pgan,4.0,600.0,tfacdiode,w * (1.0-ohmicratio),ngf,ij_pgan,0.0,vgsatq_pgan,
    I(gi2p,gi2)   <+ idsch + gmin * V(gi2p,gi2);
    if (pganrecmod == 1) begin
        idsch2    = calc_ig(idschsat2,idschrec2,vsch,phit,1.0,10.0,1.0,0.0,4.0,600.0,tfacdiode,w * (1.0-ohmicratio),ngf,0.0,0.0,vgsatq_pgan2,betarec_pgan2,irec_pgan2,pgsrec
        I(gi2p,gi2)<+ idsch2 + gmin * V(gi2p,gi2);
    end
    if (vsch <= fc * vcsh0) begin
        qsch      = type * 2.0 * csh0 * w * (1.0-ohmicratio) * ngf * vcsh0 * ( 1.0 - sqrt( 1.0 - vsch/vcsh0 ) );
    end else begin
        qsch0     = 1.0 - sqrt( 1.0 - fc );
        if (pgancshorder >= 1) begin
            qsch1c  = 1.0 / ( 2.0 * vcsh0 * sqrt(1.0-fc) );
            vschfc1 = vsch - fc * vcsh0;
            qsch1   = qsch1c * vschfc1;
            if (pgancshorder >= 2) begin
                qsch2c  = qsch1c / ( 4.0 * vcsh0 * (1.0-fc) );
                vschfc2 = vschfc1 * vschfc1;
                qsch2   = qsch2c * vschfc2;
                if (pgancshorder >= 3) begin
                    qsch3c  = qsch2c / ( 2.0 * vcsh0 * (1.0-fc) );
                    vschfc3 = vschfc2 * vschfc1;
                    qsch3   = qsch3c * vschfc3;
                    if (pgancshorder >= 4) begin
                        qsch4c  = 5.0 * qsch3c / ( 8.0 * vcsh0 * (1.0-fc) );
                        vschfc4 = vschfc3 * vschfc1;
                        qsch4   = qsch4c * vschfc4;
                        if (pgancshorder >= 5) begin
                            qsch5c  = 7.0 * qsch4c / ( 10.0 * vcsh0 * (1.0-fc) );
                            vschfc5 = vschfc4 * vschfc1;
                            qsch5   = qsch5c * vschfc5;
                        end else begin
                            qsch5c  = 0;
                        end
                    end else begin
                        qsch4c  = 0;
                    end
                end else begin
                    qsch3c  = 0;
                end
            end else begin
                qsch2c  = 0;
            end
        end else begin
            qsch1c  = 0;
        end
        qsch      = type * 2.0 * csh0 * w * (1.0-ohmicratio) * ngf * vcsh0 * ( qsch0 + qsch1 + qsch2 + qsch3 + qsch4 + qsch5 );
    end
    I(gi2p,gi2)   <+ ddt(qsch);
    if (rsch0 !=0 && ohmicratio != 0) begin
        rsch      = rsch0 / ( w * ohmicratio * ngf );
        I(gi2p,gi2)<+ V(gi2p,gi2) / rsch;
    end
```

Resistors Branch Condition

- New branch condition added to allow minr to be set to zero

```
// resistors
if (rcd_w >= minr) begin
    I(d,drc)        <+ V(d,drc) / rdi;
end else begin
    V(d,drc)        <+ 0;
end
if (rcs_w >= minr) begin
    I(src,s)        <+ V(src,s) / rsi;
end else begin
    V(src,s)        <+ 0;
end
if (rg1 <= minr) begin
    V(g,gi1)        <+ 0;
end else begin
    I(g,gi1)        <+ V(g,gi1) / rg1;
end
if (rg2 <= minr) begin
    V(gi1,gi2)      <+ 0;
end else begin




    I(gi1,gi2)      <+ V(gi1,gi2) / rg2;



end
```

```
// resistors
if ((rcd_w >= minr) && (rcd_w > 0)) begin
    I(d,drc)        <+ V(d,drc) / rdi;
end else begin
    V(d,drc)        <+ 0;
end
if ((rcs_w >= minr) && (rcs_w > 0)) begin
    I(src,s)        <+ V(src,s) / rsi;
end else begin
    V(src,s)        <+ 0;
end
if ((rg1 >= minr) && (rg1 > 0)) begin




    I(g,gi1)        <+ V(g,gi1) / rg1;




end else begin
    V(g,gi1)        <+ 0;
end
if ((rg2 >= minr) && (rg2 > 0)) begin
    I(gi1,gi2)      <+ V(gi1,gi2) / rg2;
end else begin
    V(gi1,gi2)      <+ 0;
end
```

```
    if (rcs_w>=minr) begin
        I(src,s)     <+ white_noise(4.0 * `P_KK * tdut / rsi, "rcs");
    end
    if (rcd_w>=minr) begin
        I(d,drc)     <+ white_noise(4.0 * `P_KK * tdut / rdi, "rcd");
    end
end

// power-dissipation calculations
pdiss             = ( ids * V(di,si) + idsrd * V(drc,fp4) + idsrs * V(fps4,src) + idsfps
if (rcd_w>=minr) begin
    pdiss          = pdiss + ( V(drc,d) * V(drc,d) / rdi );
end
if (rcs_w>=minr) begin
    pdiss          = pdiss + ( V(src,s) * V(src,s) / rsi );
```

```
    if ((rcs_w >= minr) && (rcs_w > 0)) begin
        I(src,s)     <+ white_noise(4.0 * `P_KK * tdut / rsi, "rcs");
    end
    if ((rcd_w >= minr) && (rcd_w > 0)) begin
        I(d,drc)     <+ white_noise(4.0 * `P_KK * tdut / rdi, "rcd");
    end
end

// power-dissipation calculations
pdiss             = ( ids * V(di,si) + idsrd * V(drc,fp4) + idsrs * V(fps4,src) + idsfps
if ((rcd_w >= minr) && (rcd_w > 0)) begin
    pdiss          = pdiss + ( V(drc,d) * V(drc,d) / rdi );
end
if ((rcs_w >= minr) && (rcs_w > 0)) begin
    pdiss          = pdiss + ( V(src,s) * V(src,s) / rsi );
```

Miscellaneous Changes & Bug Fixes

```
`MPRoz(beta,      1.50,     "",       "Linear to saturation parameter")
```

```
`MPRoz(beta,      2.0,      "",       "Linear to saturation parameter, NOTE: Users who want di
```

```
`MPRoz(rcapture, 10.0,      "Ohm",    "Trapping capture time constant")
`MPRoz(remission,50e-3,     "Ohm",    "Trapping emission time constant")
```

```
`MPRoz(rcapture, 10.0,      "Ohm",    "Trapping capture time constant associated with capture
`MPRoz(remission,50e-3,     "Ohm",    "Trapping emission time constant associated with emissi
```

```
analog function real explim;
    input x;
    real x;
    begin
        if (x>`M_MAXEXP) begin
            explim     =exp( `M_MAXEXP ) * ( 1.0 + ( x - `M_MAXEXP ));


        end else begin
            if (x<-`M_MAXEXP) begin
                explim =exp( -`M_MAXEXP );
            end else begin
                explim =exp( x );
            end
        end
    end
endfunction
```

```
analog function real explim;
    input x;
    real x;
    begin
        if (x>`M_MAXEXP) begin
            explim = exp( `M_MAXEXP ) * ( 1.0 + ( x - `M_MAXEXP ));
        end else if (x<-`M_MAXEXP) begin
            explim = exp( -`M_MAXEXP );
        end else begin
            explim = exp( x );


        end
    end
endfunction
```

```
analog function real calc_capt;
    output capout;
    input capin, tempcoin, tdutin, tnomkin;
    // IO
    real capout;
    real capin, tempcoin, tdutin, tnomkin;
    // Local
    real tcapfac;

    begin
        tcapfac      = 1.0 + tempcoin * ( tdutin - tnomkin );
        if (tcapfac  < 0.01) begin
            tcapfac   = 0.01;
        end
        capout       = capin * tcapfac;
        calc_capt    = capout;
    end
endfunction
```

```
analog function real calc_capt;

    input capin, tempcoin, tdutin, tnomkin;
    // IO

    real capin, tempcoin, tdutin, tnomkin;
    // Local
    real tcapfac;

    begin
        tcapfac      = 1.0 + tempcoin * ( tdutin - tnomkin );
        if (tcapfac  < 0.01) begin
            tcapfac   = 0.01;
        end
        calc_capt       = capin * tcapfac;

    end
endfunction
```

Miscellaneous Changes & Bug Fixes

```
if ( trapselect     == 1) begin
    // Rsh degradation with trapping
    vtcollapse0      = alphat1 * abs(V(d,g)) + explim( (V(d,g) - vttrap - V(tr1) * alphat3
    I(tr1)           <+ -vtcollapse0;
    I(tr1)           <+ V(tr1)/rintrap1;
    I(tr1,tr)        <+ ddt(ctrap * V(tr1,tr));
    I(tr)            <+ ddt(taut * V(tr));
    I(tr)            <+ V(tr);
    vtcollapse       = V(tr);
    drsht            = 1.0 + (vtcollapse) * ttrapfac;
end else if ( trapselect == 2) begin
```

```
if ( trapselect     == 1) begin
    V(dtrapin)       <+0;
    V(dtrapin2)      <+0;
    V(dtrapin3)      <+0;
    V(gtrapin)       <+0;
    V(gtrapin2)      <+0;
    V(gtrapin3)      <+0;
    // Rsh degradation with trapping
    vtcollapse0      = alphat1 * abs(V(d,g)) + explim( (V(d,g) - vttrap - V(tr1) * alphat3
    I(tr1)           <+ -vtcollapse0;
    I(tr1)           <+ V(tr1)/rintrap1;
    I(tr1,tr)        <+ ddt(ctrap * V(tr1,tr));
    I(tr)            <+ ddt(taut * V(tr));
    I(tr)            <+ V(tr);
    vtcollapse       = V(tr);
    drsht            = 1.0 + (vtcollapse) * ttrapfac;
end else if ( trapselect == 2) begin
    V(tr1)           <+0;
    V(tr)            <+0;
```

```
// noise calculations
gm                   = 0;
svc                  = 0;
if (noisemod == 1) begin
    I(gi2p,si)       <+ white_noise(shs * `P_QQ * abs(igsi + 2.0 * (igssdio + igsrec)), "
    I(gi2p,di)       <+ white_noise(shd * `P_QQ * abs(igdi + 2.0 * (igdsdio + igdrec)), "
```

```
// noise calculations
gm                   = 0;
svc                  = 0;
if (noisemod == 1) begin
    I(gi2p,si)       <+ white_noise(shs * `P_QQ * abs(igsidb + 2.0 * (igssdiodb + igsrecdb
    I(gi2p,di)       <+ white_noise(shd * `P_QQ * abs(igdidb + 2.0 * (igdsdiodb + igdrecdb

    I(gi2p,fps4)     <+ white_noise(shs * `P_QQ * abs(igsi + 2.0 * (igssdio + igsrec)), "
    I(gi2p,fp4)      <+ white_noise(shd * `P_QQ * abs(igdi + 2.0 * (igdsdio + igdrec)), "
```

V4.0.0 VAMPyRE (1.9.3) Test Result

```
$ python vampyre.py mvsg_cmc_4.0.0.va -a -f
Reading mvsg_cmc_4.0.0.va
Reading disciplines.vams
STYLE in file disciplines.vams, line 1: MS-DOS file format (\r\n)
Reading constants.vams
WARNING in file mvsg_cmc_4.0.0.va, line 1821: Call to ddx(), but ids does not depend on V(b)
WARNING in file mvsg_cmc_4.0.0.va, line 1825: Call to ddx(), but qgi does not depend on V(b)
WARNING in file mvsg_cmc_4.0.0.va, line 1829: Call to ddx(), but qdi does not depend on V(b)
WARNING in file mvsg_cmc_4.0.0.va, line 1833: Call to ddx(), but qsi does not depend on V(b)
WARNING in file mvsg_cmc_4.0.0.va, line 1834: Call to ddx(), but qbi does not depend on V(gi2p)
WARNING in file mvsg_cmc_4.0.0.va, line 1835: Call to ddx(), but qbi does not depend on V(di)
WARNING in file mvsg_cmc_4.0.0.va, line 1836: Call to ddx(), but qbi does not depend on V(si)
WARNING in file mvsg_cmc_4.0.0.va, line 1837: Call to ddx(), but qbi does not depend on V(b)
WARNING in file mvsg_cmc_4.0.0.va, line 120: Parameter 'version' was never used

Module: mvsg_cmc

Instance parameters (4):
   dtemp, l, ngf, w

Model parameters (352)

Operating-point variables (38):
   cbbi, cbdi, cbgi, cbsi, cdbi, cddi, cdgi, cdsi, cgbi, cgd, cgdi,
   cggi, cgs, cgsi, csbi, csdi, csgi, cssi, gdsi, gmbsi, gmi, idisi,
   igd, igs, pdc, qbi, qdi, qgi, qsi, rd, rs, t_delta_sh, t_total_c,
   t_total_k, vdisi, vdsati, vgisi, vti
```